

GODOT:

PADRÕES DE COMMIT E BRANCHES

PARA GAME DEVELOPMENT

Introdução

Autor: Yuji

Grupo: Obscura

Esse documento tem como objetivo apresentar os **padrões de commit** e **branches** utilizados no mercado de trabalho, com foco na área de **Game Development** (desenvolvimento de jogos), além de trazer dicas para o versionamento de código no **GitHub** e exemplos práticos aplicados pelo autor.

O documento é uma pesquisa de um dos integrantes do grupo de pesquisa e desenvolvimento de jogos, **Obscura**.

Commits sem Padrões

É muito comum para iniciantes da área de computação não saberem como realizar o versionamento de código da maneira correta, principalmente para os estudantes. É comum ver commits feitos direto na branch principal ou terem nomes despadronizados, como:

- Update
- Teste
- Arrumei bug
- Coloquei método

Essas mensagens dificultam o entendimento e organização do histórico, identificação de mudanças e colaboração em equipe.

O que são padrões de commit?

Os padrões de commit são convenções utilizadas para padronizar mensagens no **Git**, permitindo um histórico mais organizado e compreensível.

Segundo a documentação oficial do **Conventional Commits**, os **commits semânticos** são uma convenção simples para descrever claramente a intenção de cada commit.

Essa convenção define uma série de regras para criar um histórico de commit claro e organizado, facilitando a criação de ferramentas automatizadas e alteração do código.

Essa identificação ocorre através de uma palavra e emoji que identificam se o commit se trata de uma modificação, atualização de pacotes, documentação e entre outros. Para mais informações sobre os emojis, confira:

<https://github.com/iuricode/padroes-de-commits.git>

Por que utilizar padrões de commit na Game Dev?

No desenvolvimento de jogos, especialmente quando há trabalho em equipe, há a separação de diferentes tarefas entre os membros do grupo que precisam ser gerenciadas e organizadas corretamente.

Além disso, temos diferentes partes do jogo a serem desenvolvidas ao longo de todo o processo como:

- **Mecânicas:** player, inimigos, ataques;
- **Assets:** sprites, áudios;
- **Sistemas:** física, Interface do Usuário, câmera.

Sem os padrões de commit, teremos um histórico confuso, dificuldade em rastrear erros e confusão entre a equipe.

Estrutura do Commit

Um commit pode ser dividido em três partes principais:

Cabeçalho

tipo (escopo): descrição

- **Tipo:** indica a intenção do commit.
- **Escopo:** especifica qual parte do projeto está sendo alterada.

Corpo (opcional): descrição detalhada do commit.

Rodapé (opcional): é utilizado para adicionar informações extras, como:

- Referências a issues (Issues #ID);
- Revisores (Reviewed By);
- Breaking changes (BREAKING CHANGE);
- Links ou tarefas relacionadas;
- Resolução de issues (Closes #ID).

Boas Práticas de Commits

- Commits pequenos;
- Escreva mensagens claras e descritivas;
- Use verbos no início do commit.

Issues no GitHub

As **Issues** no **GitHub** são ferramentas utilizadas para **registrar tarefas, bugs, melhorias** ou **discussões** dentro do projeto. Funcionam como um sistema de **organização** e **acompanhamento** do desenvolvimento.

São essenciais para a **game dev**, já que podemos organizar mecânicas, UI, assets e sistemas.

Estrutura da Issue

Uma **Issue** geralmente possui:

- **Título:** resumo curto da tarefa.
 - **Ex:** Bug: player atravessa plataforma
- **Descrição:** explicação detalhada.
 - **Ex:**
O player está atravessando plataformas ao cair de grandes alturas.

Passos para reproduzir:
1. Pular de uma altura alta
2. Cair sobre a plataforma

Resultado esperado:
O player deve colidir corretamente com a plataforma.
- **Labels (rótulos):** classificam a Issue.
 - **Ex:** bug, enhancement, ui, gameplay
- **Assignee (assinatura):** responsável pela tarefa.
- **Número da Issue:** cada Issue recebe um ID
 - **Ex:** Issues #15

Para criar uma Issue no repositório GitHub, basta clicar em:

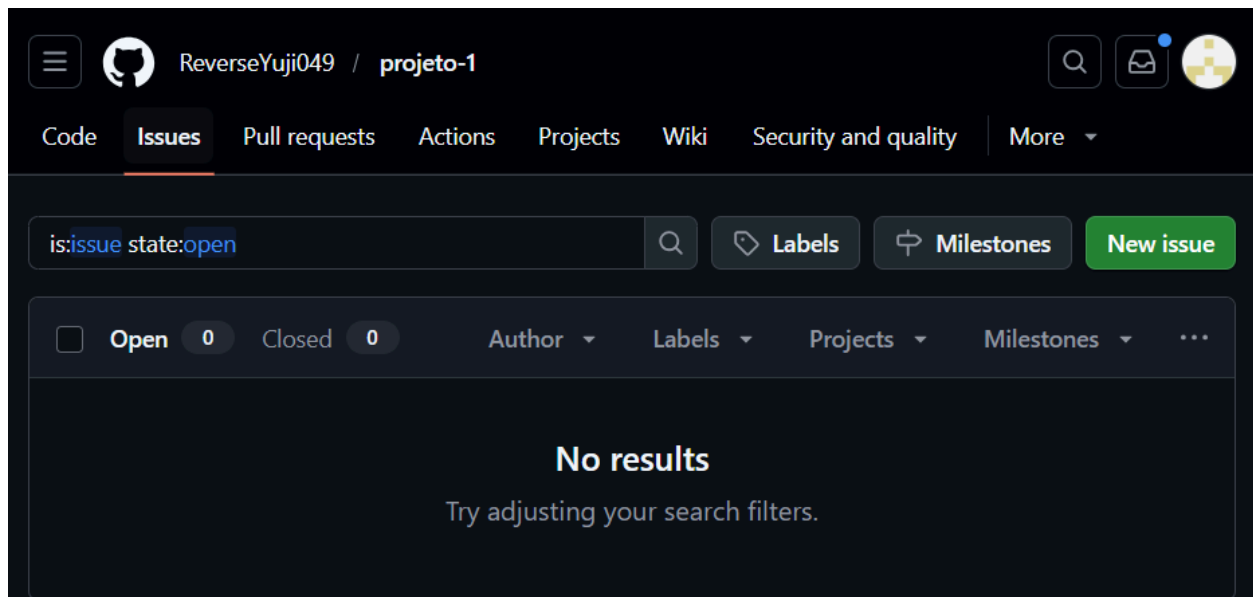
Issues → New Issue → Preencha as seções da Issue

Integração com Issues

O **GitHub** permite vincular **commits** com **issues**. Por exemplo:

fix(player): corrige colisão
Closes #15

Isso significa que o commit resolve a issue 15, que será automaticamente fechada com a branch.



Boas Práticas de Issues

- Crie uma Issue para cada tarefa
- Use títulos claros e descritivos
- Descreva bem o problema ou tarefa
- Relacione os commits e Pull Requests
- Não use Issues genéricas
- Não esqueça da descrição

Tipos de Commit

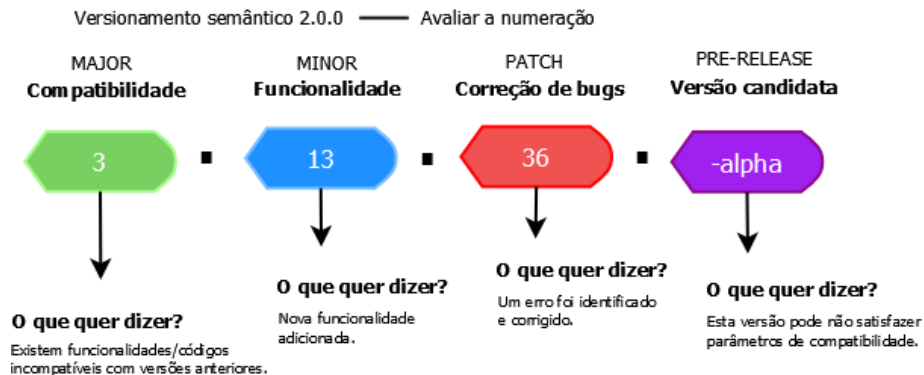
- **feat:** nova funcionalidade/recurso
 - **Ex:** feat(player): adiciona dash
- **fix:** correção de bug/erro
 - **Ex:** fix(camera): corrige tremor
- **docs:** adiciona guia do sistema de combate
 - **Ex:** docs: adiciona guia do sistema de combate
- **test:** testes do projeto
 - **Ex:** test(player): testa sistema de movimento
- **build:** dependências/build
 - **Ex:** build: adiciona plugin de áudio
- **perf:** performance
 - **Ex:** perf(render): otimiza FPS
- **style:** formatação/visual
 - **Ex:** style(ui): melhora HUD
- **refactor:** refatoração sem alterar o comportamento
 - **Ex:** refactor(movement): reorganiza código
- **chore:** configuração geral
 - **Ex:** chore: atualiza versão do Godot
- **ci:** Integração Contínua
 - **Ex:** ci: adiciona pipeline da build
- **cleanup:** limpeza do código
 - **Ex:** cleanup: remove código morto
- **remove:** remoção de arquivos
 - **Ex:** remove: exclui sprites antigos

Os tipos de commit ajudam a identificar rapidamente o que foi feito no projeto e sua intenção de alteração/atualização.

Versionamento Semântico

O versionamento semântico (**Semantic Versioning - SemVer**) é uma convenção utilizada para definir **versões de software** de forma padronizada, facilitando o controle de mudanças e a comunicação entre desenvolvedores.

Ele segue a seguinte estrutura:



Fonte da imagem: <https://matteus.dev/versionamento-semantico-2-0-0/>

O versionamento, por exemplo, v1.2.0, é usado nas **Releases**, no lançamento do projeto, no **GitHub**. A versão representa uma versão oficial e um ponto importante do projeto. Não utilize-as nas branches e commits.

Tags do Git

As versões também são usadas como tags:

Git Bash:

- git tag v1.2.0
- git push origin v.1.2.0

Isso “marca” o commit como uma **versão do projeto**.

Relações com Commits

- **feat** → **MINOR**: Commits do tipo **feat** indicam adição de novas funcionalidades:
 - **Exemplo:** 1.3.0 → 1.4.0
- **fix** → **PATCH**: Commits do tipo **fix** indicam correções de bug:
 - **Exemplo:** 1.4.0 → 1.4.1

- **BREAKING CHANGE** → **MAJOR**: quando uma alteração quebra a funcionalidade:
 - **Ex:**
feat(movement): altera sistema de movimentação

BREAKING CHANGE: novo sistema incompatível com versões anteriores

1.4.1 → 2.0.0

Os outros tipos de commit não impactam diretamente a versão.

Exemplos

Exemplo 1: Adição de mecânica

- **Cabeçalho:** feat(player): adiciona sistema de pulo duplo
- **Descrição:**
 - Implementado sistema de pulo duplo para o player.
 - Adiciona variável de controle de salto
 - Verificação se o player está no ar
 - Reset ao tocar no chão

Melhora a mobilidade do personagem durante a gameplay.

Exemplo 2: Correção

- **Cabeçalho:** fix(player): corrige bug ao atravessar plataformas
- **Descrição:**

Corrigido o erro onde o player atravessava plataformas ao cair.

 - Ajustada detecção de colisão
 - Corrigido o cálculo de velocidade vertical

- **Rodapé:**

Closes #15

Reviewed-by: Yuji

Exemplo 3: Refatoração

- **Cabeçalho:** refactor(movement): reorganiza lógica de movimentação
- **Descrição:**
 - Refatorada a lógica de movimentação do player.
 - Separação de funções menores
 - Remoção de código duplicado

Nenhuma mudança no comportamento do jogo.

Exemplo 4: UI

- **Cabeçalho:** style(ui): melhora HUD de vida
- **Descrição:**
 - Atualizado o layout da barra de vida.
 - Ajustado posicionamento
 - Melhor contraste de cores
 - Adicionada animação ao perder vida

Padrões de Branches

Branches são **ramificações** de seu código principal que permitem o desenvolvimento de funcionalidades, correções e testes sem afetar diretamente a versão estável do produto.

Em projetos de jogos, várias coisas acontecem simultaneamente durante o desenvolvimento, especialmente em equipes. Por exemplo:

- Um programador mexe no player.
- Outro cria inimigos.
- Outra melhora a HUD (Heads-Up Displays) do Player na UI (Interface do Usuário).
- Outra arruma a física do jogo.

Sem os padrões das branches, o seu código pode acabar sendo sobrescrito, duplicado, pode gerar conflitos constantes, como conflitos de merge (**merge conflicts**) e dificuldade em compreender e organizar o código.

Estrutura das Branches

tipo/nome-da-feature

Para nomear suas branches, utilize o **kebab-case** (com hífen) e seja claro e específico.

Organização

A estrutura mais recomendada é:

- **main**: versão final e estável (produção)
- **develop**: versão em desenvolvimento
- **branches auxiliares**: utilizadas para desenvolvimento de funcionalidades, correções e melhorias (features, fixes, refactor, etc.)

Faça uma branch para uma responsabilidade única, e não mais de uma como "feat/player-and-ui-and-audio"; isso pode causar confusão e má organização de seu código.

Boas Práticas

- Crie branches pequenas
- Dê nomes claros e específicos
- Sempre atualize com **develop**
- Exclua a branch após o **merge** com o **develop** ou **main**

Pull Request

O **Pull Request** é utilizado para revisar e integrar alterações ao projeto principal (branch main). Ela precisa ter:

- Título padronizado
- Descrição clara
- Passos para teste
- Referência a Issues

Tipos de Branches

- **feat:** nova funcionalidade
 - **Ex:** feat/player-movement
- **fix:** correção de bug/erro
 - **Ex:** fix/collision-bug
- **test:** testes do projeto
 - **Ex:** test/player-mechanics
- **perf:** performance
 - **Ex:** perf/render-optimization
- **style:** formatação/visual
 - **Ex:** style/hud-update
- **refactor:** refatoração sem alterar o comportamento
 - **Ex:** refactor/movement-system
- **chore:** configuração geral
 - **Ex:** chore/update-godot-version

Fluxo de Branches

1. Atualize a branch develop:
 - **Git Bash:**
 - git pull origin develop
2. Crie uma branch a partir da **develop**:
 - **Git Bash:**
 - git checkout develop
 - git checkout -b feat/player-jump
3. Desenvolva a funcionalidade e faça o **commit** padronizado:
 - feat(player): adiciona pulo
4. Envie para o **GitHub**:
 - **Git Bash:**
 - git push origin feat/player-jump
5. Crie o **Pull Request**:

Exemplo:

- **Título:** feat(player): adiciona sistema de pulo duplo
- **Descrição :**
 - O que foi feito:
 - Implementado sistema de pulo duplo
 - Adiciona verificação de chão
 - Impacto no jogo:

- Melhora a mobilidade do player:

Como testar:

- Pressione espaço duas vezes

Referências

- Issue #12

6. Revise o código

7. Faça **Merge** na **develop**

Adaptação para a Game Dev

Para o desenvolvimento de jogos, recomenda-se utilizar os seguintes **escopos**:

- player
- enemy
- ui
- audio
- level
- camera
- physics
- animation
- input
- combat

Exemplo de fluxo:

- **feat(player)**: adiciona pulo duplo
- **fix(player)**: corrige bug ao cair da plataforma
- **feat(enemy)**: adiciona IA básica
- **perf(render)**: melhora desempenho de partículas.

Aplicação Prática

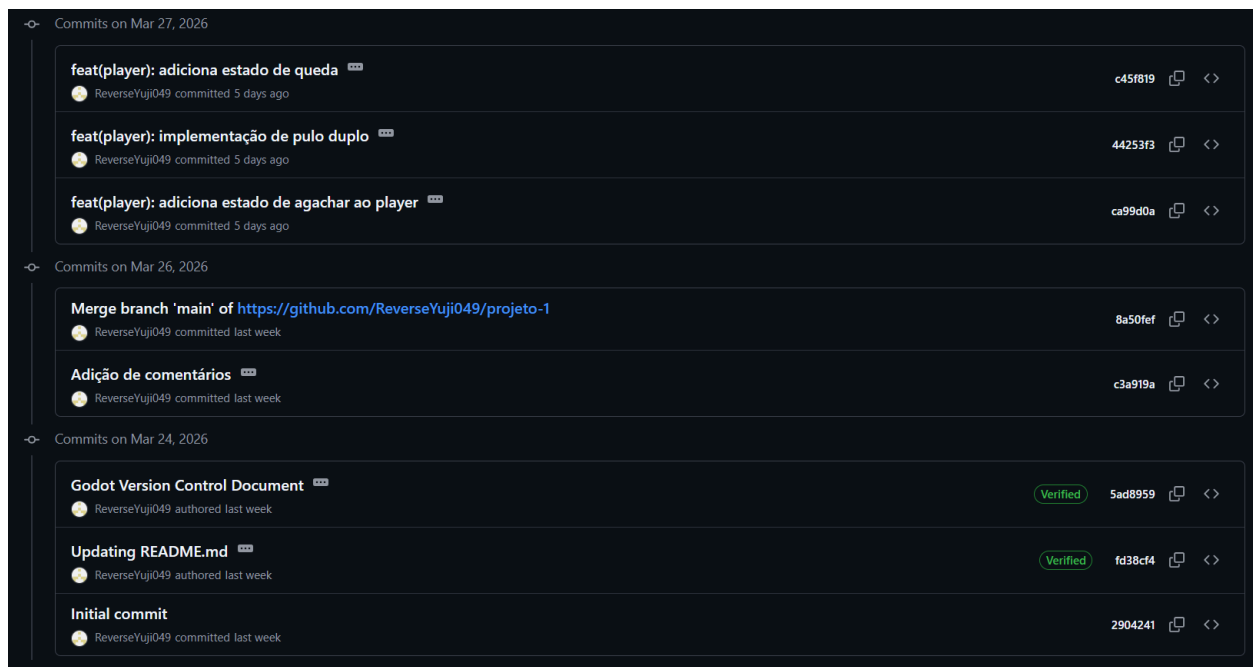
Repositório da aplicação: <https://github.com/ReverseYuji049/projeto-1.git>

Overview	Yours	Active	Stale	All
Q Search branches...				
Default				
Branch	Updated	Check status	Behind / Ahead	Pull request
main	last week		Default	...
Your branches				
Branch	Updated	Check status	Behind / Ahead	Pull request
feature/player-fall-status	5 days ago		0 3	...
feature/double-jump	5 days ago		0 2	...
feature/player-duck-status	5 days ago		0 3	...
Active branches				
Branch	Updated	Check status	Behind / Ahead	Pull request
feature/player-fall-status	5 days ago		0 3	...
feature/double-jump	5 days ago		0 2	...
feature/player-duck-status	5 days ago		0 3	...

Na imagem, apresenta uma lista de branches:

- A **main** é a branch “**default**”, a versão estável e final.
- A **feature/player-fall-status** é a adição do estado do Player ao cair.
- A **feature/double-jump** é a adição da mecânica de pulo duplo do Player.
- A **feature/player-duck-status** é a adição do estado do Player ao agachar.

Em cada branch, é possível ver o histórico de commits em ordem cronológica, desde a main até a branch atual:



O versionamento do código foi feito no **GitHub Desktop**. Caso tenha dúvidas sobre o versionamento do Godot, aqui está minha pesquisa sobre o assunto:

- **Slides (PDF):**
https://drive.google.com/file/d/1f0Gfoym-TanzV5ncJ7E_UJRuav9BTlZQ/view?usp=sharing
- **Documento (PDF):**
https://drive.google.com/file/d/1EcGzzQb1r2MrC8ToEzOwGeZwK2STps0T/view?usp=drive_link

Conclusão

A utilização de **padrões de commit** e **branches** contribui diretamente para a **organização, compreensão, rastreabilidade** e **qualidade** do desenvolvimento de software.

Na área de **Game Development**, onde múltiplos sistemas são desenvolvidos simultaneamente, essa padronização se torna essencial para o **trabalho em equipe** e **manutenção do projeto**.

Além disso, o uso de **Issues** no **GitHub** permite um melhor **gerenciamento de tarefas**, possibilitando o **registro de bugs**, **funcionalidades** e **melhorias**, bem como a **organização do fluxo de trabalho da equipe**. A **integração** entre **Issues**, **commits** e **Pull Requests** garante **maior controle** e **visibilidade** sobre o desenvolvimento do projeto.

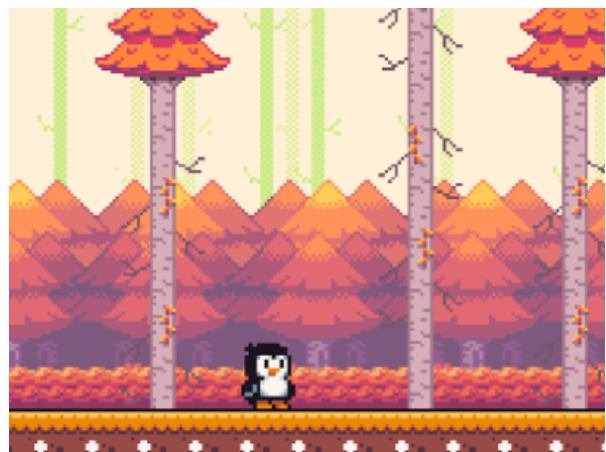
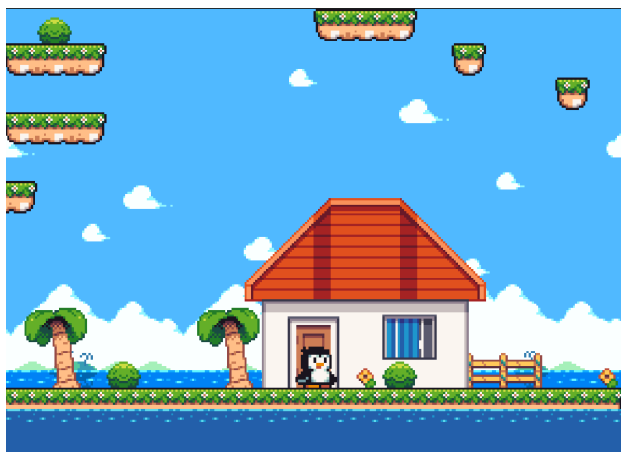
Outro ponto importante é o **versionamento semântico**, que permite **classificar** e **organizar** as **versões do software** de forma clara e padronizada. A relação entre os **tipos de commit** (como feat e fix) e os **incrementos de versão** (MINOR e PATCH) facilita a **compreensão das mudanças realizadas a cada atualização**, além de possibilitar **automações** no processo de **release**.

Referências

- **GitHub:** <https://github.com/iuricode/padroes-de-commits.git>
- **Documentação Conventional Commits:**
<https://www.conventionalcommits.org/pt-br/v1.0.0/>
- **Versionamento semântico:**
<https://matteus.dev/versionamento-semantico-2-0-0/>

Veja Também

Repositório contendo estudos de Godot através do curso no Youtube do professor **Rafael Forbeck**: <https://github.com/ReverseYuji049/projeto-1.git>



Meu portfólio do Pensamento Computacional:

<https://sites.google.com/edu.unifil.br/pensamentocompyuji/p%C3%A1gina-inicial>



Google Drive com pesquisas e documentos do grupo Obscura:

https://drive.google.com/drive/folders/1uBIOD5ariPldJ1IzlnMDFczkXortsWxQ?usp=drive_link

